

A Multi-Agent LLM Architecture with Dynamic Cascading for On-Premises, LGPD-Compliant Organizational Assistants

Renato de Oliveira Rosa

Fucape Business School – Espírito Santo – ES – Brazil

gestor.renatorosa@gmail.com

Abstract. *The integration of large language models (LLMs) into data-sensitive organizational environments, such as legal departments, public agencies, healthcare providers, financial services, and notary offices, faces two structural barriers: regulatory constraints on data processing, exemplified in Brazil by the Lei Geral de Proteção de Dados (LGPD, Law 13.709/2018), and the heterogeneity of user workloads at the edge. This paper presents a multi-agent LLM architecture that combines full on-premises execution with a dynamic cascading router, with the objective of providing LGPD-compliant AI assistance to any organization that handles personal or confidential data. The system is composed of cooperating agents: a heuristic router agent, four specialized LLM agents (Tier 1 ultra-light, Tier 2 intermediate, Tier 3-Reasoning, and Tier 3-Code), a per-workspace retrieval-augmented generation (RAG) agent, a compliance and audit agent, and a real-time performance agent. The router dispatches each incoming query to the cheapest viable specialized LLM agent and transparently cascades to lighter tiers when the primary agent fails because of timeouts, out-of-memory errors, or network unavailability. This paper describes the architecture, the cascading protocol with empirical latency distributions, the LGPD masking pipeline, and four reproducible demonstration scenarios that will be executed live at the conference. The system is fully functional, packaged as a single Windows executable, and is currently in production use at the deploying institution.*

Resumo. *A integração de modelos de linguagem de grande porte (LLMs, do inglês Large Language Models) em ambientes organizacionais que tratam dados sensíveis, como departamentos jurídicos, órgãos públicos, prestadores de serviços de saúde, instituições financeiras e cartórios, enfrenta duas barreiras estruturais: as restrições regulatórias sobre o tratamento de dados, exemplificadas no Brasil pela Lei nº 13.709/2018 (Lei Geral de Proteção de Dados Pessoais, LGPD), e a heterogeneidade das cargas de trabalho na borda. Este trabalho apresenta uma arquitetura multi-agente de LLMs que combina a execução totalmente local com um roteador dinâmico em cascata, com vistas a oferecer assistência de IA em conformidade com a LGPD a qualquer organização que trate dados pessoais ou confidenciais. O sistema é composto por agentes cooperantes: um agente roteador heurístico, quatro agentes LLM especializados (Tier 1 ultra-leve, Tier 2 intermediário, Tier 3-Raciocínio e Tier 3-Código), um agente de geração aumentada por recuperação (RAG, do inglês Retrieval-Augmented Generation) isolado por workspace, um agente de auditoria e conformidade, e um agente de monitoramento de desempenho em tempo real. O roteador despacha cada consulta ao agente LLM especializado mais barato capaz de respondê-la e realiza fallback transparente para tiers*

mais leves quando o agente primário falha por esgotamento de tempo, falta de memória ou indisponibilidade de rede. Neste trabalho, descrevemos a arquitetura, o protocolo de cascata com distribuições empíricas de latência, o pipeline de mascaramento de dados pessoais e quatro cenários reprodutíveis de demonstração que serão executados ao vivo no congresso. O sistema encontra-se em pleno funcionamento, empacotado como um único executável para Windows, e em uso produtivo na instituição implantadora.

1. Introdução

A evolução dos modelos de linguagem de grande porte tem promovido transformações relevantes em diferentes setores produtivos (Russell and Norvig, 2021). No entanto, a incorporação desses modelos em ambientes organizacionais que tratam dados sensíveis esbarra em entraves que vão além do desempenho técnico. Por um lado, regulações de proteção de dados, como a Lei nº 13.709/2018 no Brasil (Brasil, 2018), o Regulamento Geral de Proteção de Dados da União Europeia (GDPR, do inglês General Data Protection Regulation; Conselho da União Europeia, 2016) e a California Consumer Privacy Act (CCPA; Estado da Califórnia, 2018), impõem requisitos rigorosos ao tratamento de dados pessoais, o que inviabiliza, em grande parte dos cenários práticos, o envio de consultas e documentos a provedores remotos de LLM (frequentemente operados por corporações norte-americanas ou asiáticas).

Por sua vez, essa submissão de dados a APIs de terceiros não apenas fere os princípios de confidencialidade e consentimento, mas também cria gargalos de soberania de dados e dependência de fornecedores externos (vendor lock-in). Por outro lado, a heterogeneidade das demandas dos usuários de mesa, que alternam entre saudações triviais, consultas a políticas internas, análises técnicas aprofundadas e geração de código, torna economicamente desfavorável a adoção de um único modelo de grande porte para todas as consultas.

Nesse contexto, o presente trabalho apresenta uma arquitetura multi-agente de LLMs executada integralmente em ambiente local (on-premises) e concebida para servir como assistente corporativo em qualquer organização que necessite conciliar soberania de dados, conformidade regulatória e eficiência computacional. Para tanto, a solução articula um conjunto de agentes cooperantes em torno de um roteador dinâmico em cascata, com o objetivo de conciliar a soberania de dados exigida por regulações como a LGPD com a eficiência computacional demandada por cargas de trabalho heterogêneas. De modo específico, Em sua composição, a arquitetura combina um agente roteador heurístico, quatro agentes LLM especializados por capacidade, um agente de RAG isolado por workspace, um agente de auditoria com mascaramento de dados sensíveis e um agente de monitoramento de desempenho com transmissão contínua por Server-Sent Events (SSE).

A generalidade da proposta é evidenciada pelos cenários de demonstração, que contemplam casos de uso em domínios diversos, como consulta a regulamentos internos, comparação entre regimes jurídicos, geração de código para validação de dados, recuperação de políticas de recursos humanos e pesquisa em bases técnicas. Embora o sistema tenha sido implantado em uma primeira organização de referência, seu projeto não assume qualquer domínio de aplicação específico e pode ser reaproveitado em

diferentes contextos institucionais mediante a configuração adequada dos workspaces e da taxonomia de palavras-chave do roteador.

O presente demo paper está organizado em sete seções, além desta introdução. Em primeiro lugar, a Seção 2 situa a proposta na literatura de sistemas multi-agente, agentes baseados em LLM e cascadeamento de modelos. Na sequência, a Seção 3 descreve a arquitetura multi-agente. A Seção 4, por seu turno, detalha sua implementação. Logo após, a Seção 5 apresenta os quatro cenários reprodutíveis que serão executados ao vivo no congresso. A Seção 6, por sua vez, discute observações empíricas. Por fim, a Seção 7 sintetiza as contribuições e aponta direções de trabalho futuro.

2. Trabalhos Relacionados

A presente proposta articula três linhas de pesquisa consolidadas: sistemas multi-agente, agentes baseados em LLM e cascadeamento de modelos. Esta seção revisita cada uma delas, com vistas a delimitar a contribuição específica da arquitetura descrita.

2.1 Sistemas multi-agente

A área de sistemas multi-agente (SMA) possui tradição consolidada no estudo de entidades computacionais autônomas que percebem seu ambiente e agem sobre ele de forma a atingir objetivos individuais ou coletivos (Wooldridge, 2009). A título de exemplo, arquiteturas clássicas, como o modelo BDI (Belief-Desire-Intention) proposto por Rao e Georgeff (1995), formalizam o raciocínio prático do agente em termos de crenças, desejos e intenções, e servem de referência para linguagens de programação de agentes como o Jason (Bordini et al., 2007).

Ademais, a literatura também investiga a coordenação entre agentes por meio de negociação, argumentação e reputação, conforme sintetizado em estudos de revisão (Dignum, 2004). Sobretudo, os SMA oferecem repertório conceitual para descrever sistemas em que múltiplos componentes com responsabilidades especializadas cooperam em prol de um objetivo comum, característica que o presente trabalho preserva ao modelar a arquitetura como uma sociedade de agentes LLM especializados.

2.2 Agentes baseados em LLM

Nos últimos anos, a literatura tem explorado o uso de LLMs como política interna de agentes, substituindo ou complementando módulos simbólicos tradicionais. Recentemente, o paradigma Chain-of-Thought (Wei et al., 2022) demonstrou que a eliciação de passos intermediários de raciocínio melhora o desempenho em tarefas complexas, e o ReAct (Yao et al., 2023) combina raciocínio e ação por meio do uso de ferramentas externas.

Nessa direção, trabalhos como o Toolformer (Schick et al., 2023) e os Generative Agents (Park et al., 2023) ampliam essa linha ao permitir que agentes LLM invoquem APIs, executem código e interajam entre si em ambientes simulados. De forma complementar, Mialon et al. (2023) sistematizam o campo dos Augmented Language Models e discutem caminhos para a ampliação das capacidades dos modelos por meio de

mecanismos externos. Diante disso, a arquitetura descrita neste trabalho se inscreve nessa vertente ao tratar cada tier de LLM como um agente especializado capaz de responder a um subconjunto de consultas, sob a coordenação de um agente roteador.

2.3 Cascadeamento e roteamento de modelos

A literatura sobre cascadeamento de modelos investiga como distribuir consultas de complexidade variável entre modelos de custo distinto, com vistas a reduzir o custo agregado sem perda significativa de qualidade. Em uma vertente correlata, Schuster et al. (2021) introduziram o Confident Adaptive Language Modeling (CALM), que desacelera a decodificação em partes críticas do texto.

De forma complementar, Chen et al. (2023), no FrugalGPT, demonstram que cadeias de modelos de portes variados podem ser orquestradas para reduzir custos em ordens de magnitude. Em paralelo, Ding et al. (2024) propõem abordagens híbridas em que a complexidade da consulta determina o modelo invocado. Em comum, essas abordagens tratam a decisão de roteamento como um problema de classificação cuja resposta define o modelo a ser acionado. Por fim, o presente trabalho adota esse mesmo princípio, mas o implementa como um agente roteador explícito, observável e thread-safe, cujo comportamento é exposto ao usuário por meio de um indicador visual no frontend.

2.4 Geração aumentada por recuperação

A técnica de RAG, introduzida por Lewis et al. (2020), combina um mecanismo de recuperação de documentos com um gerador neural, com o objetivo de fundamentar as respostas do modelo em fontes externas verificáveis. Nessa perspectiva, Gao et al. (2023) revisitam o tema em survey recente, na qual discutem arquiteturas, mecanismos de indexação e métricas de avaliação. Diante disso, a arquitetura descrita neste trabalho incorpora um agente de RAG cuja base vetorial é isolada por workspace, de modo a evitar o vazamento de informações entre domínios de conhecimento.

2.5 Inferência local e conformidade regulatória

O debate sobre a execução local de LLMs tem sido impulsionado por requisitos de soberania de dados, confidencialidade e custo. Sob esse prisma, a literatura aponta que a inferência em borda (edge inference) é viável com modelos de código aberto servidos por runtimes como o Ollama (Ollama, 2024) e combinada a frameworks de orquestração como o LangChain (LangChain, 2024).

No contexto regulatório brasileiro, a LGPD estabelece bases legais para o tratamento de dados pessoais e impõe obrigações de registro, segurança e transparência (Brasil, 2018). Dessa forma, o presente trabalho situa a arquitetura descrita na interseção entre essas duas agendas, ao propor um sistema multi-agente capaz de operar de forma totalmente local e em conformidade com a legislação vigente em diferentes jurisdições.

3. Arquitetura Multi-Agente

A arquitetura é descrita nesta seção como uma sociedade de agentes cooperantes, cada um implementado em um módulo específico. A Figura 1 apresenta a visão geral do sistema.

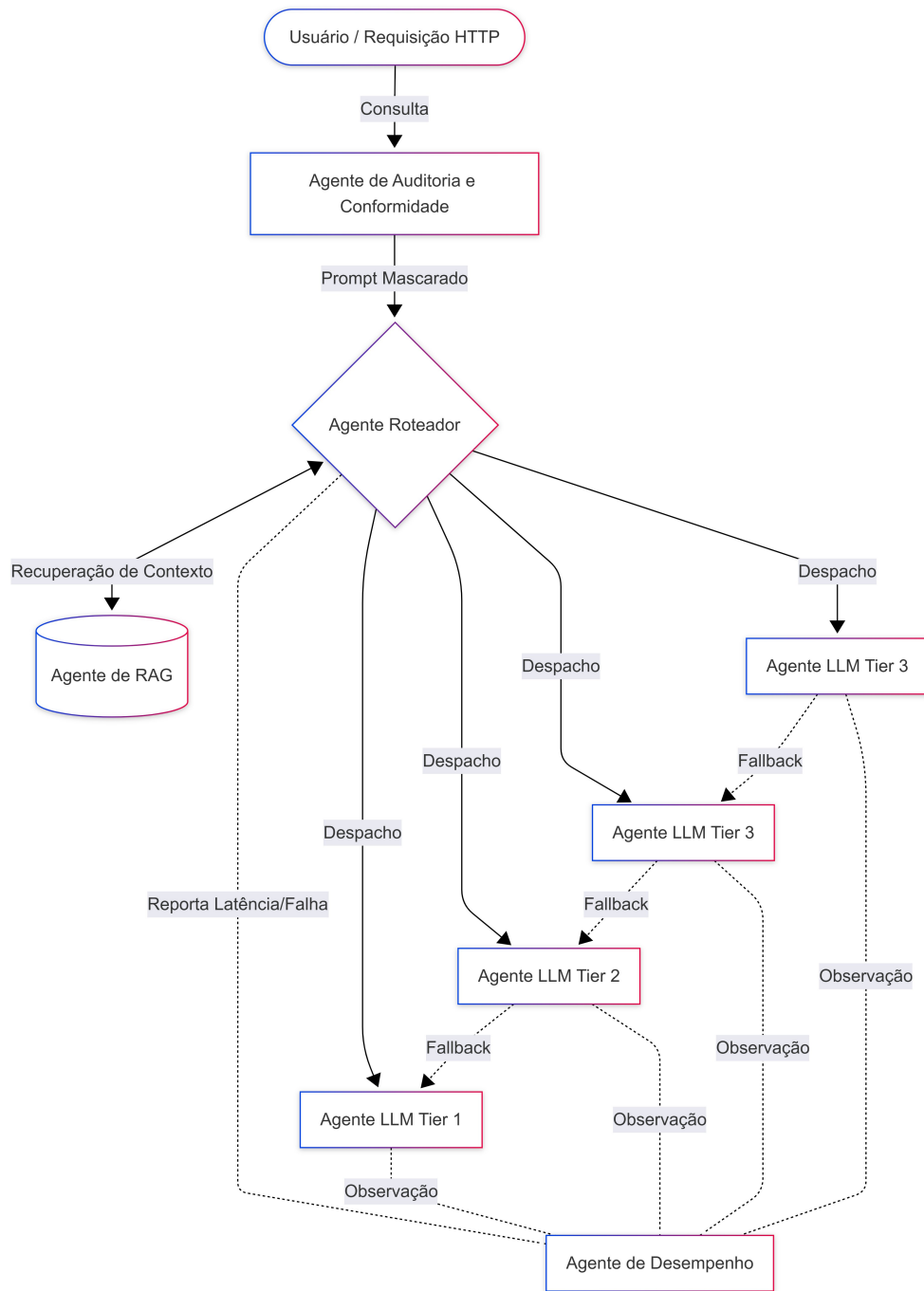


Figura 1. Visão geral da arquitetura, em que o agente roteador despacha cada consulta a um dos quatro agentes LLM especializados, enquanto os agentes de RAG, auditoria e desempenho observam todas as interações.

3.1 Agente roteador

O agente roteador, implementado no módulo `llm_router.py`, expõe a primitiva `classify(prompt, rag_context, requested_model)`, que retorna uma decisão de roteamento contendo o tier selecionado, a justificativa da escolha, o tamanho do contexto em caracteres e a cadeia de fallback correspondente. Para tanto, a decisão é construída em quatro etapas sequenciais.

Na primeira etapa, verifica-se se o usuário solicitou um modelo específico por meio do menu de configuração do frontend. Em caso afirmativo, o roteador respeita a solicitação e apenas monta a cadeia de fallback. Na segunda etapa, um regex compilado identifica sinais de alta complexidade na consulta, abrangendo palavras-chave relacionadas a código (por exemplo, `python`, `script`, `regex`, `sql`, `api`, `json`, `algoritmo`, `debug`), a cálculo (por exemplo, `calcule`, `derivada`, `estatística`, `probabilidade`), a temas fiscais e tributários (por exemplo, `regime tributário`, `ICMS`, `ISS`, `SPED`, `CFOP`, `simples nacional`, `lucro real`, `lucro presumido`) e a raciocínio jurídico (por exemplo, `lei`, `artigo`, `jurisprudência`, `contrato`, `análise crítica`, `compare`, `diferença entre`). Na terceira etapa, o tamanho agregado do prompt e do contexto RAG é comparado a dois limiares: quando ultrapassa 6.000 caracteres, a consulta é encaminhada ao Tier 3; quando ultrapassa 2.000 caracteres, ao Tier 2; caso contrário, permanece no Tier 1. Por fim, na quarta etapa, o Tier 1 efetivo é escolhido de forma adaptativa, com base nos modelos efetivamente disponíveis no Ollama local, em uma lista ordenada do mais leve para o mais pesado.

O estado interno do roteador é protegido por um lock de módulo, e a invocação dos modelos é serializada por um lock específico para evitar contenção de GPU e CPU.

3.2 Protocolo de cascata com fallback

O protocolo de cascata é implementado em `llm_router.call_with_fallback`. Para cada tier presente na cadeia de fallback, o método `_try_model` registra a latência, o código HTTP retornado pelo Ollama e, conforme o caso, o texto gerado ou uma mensagem de erro estruturada. Nessa lógica, o primeiro tier que devolver resposta não vazia é considerado o vencedor. Caso todos os tiers falhem, o sistema devolve uma mensagem de erro controlada, que é igualmente registrada no log de auditoria.

A ordem canônica da cadeia é `[TIER_3_CODE, TIER_3_REASON, TIER_2, TIER_1]`, em consonância com o princípio de tentar inicialmente o modelo mais capaz disponível e, em seguida, modelos progressivamente mais leves. Tal protocolo torna o sistema resiliente, na medida em que a falha de um agente é observada pelo orquestrador e tratada por meio da cooperação entre os demais agentes, em vez de se traduzir em erro visível ao usuário.

3.3 Agente de RAG

O agente de RAG, implementado em `workspaces.py`, materializa cada workspace como uma coleção do ChromaDB (Chroma, 2024) no diretório `./workspaces_db/{slug}/`, acompanhada de um arquivo `manifest.json` com metadados descritivos. Internamente, o agente é responsável por (a) ingerir documentos PDF por meio do `PyPDFLoader` e do `RecursiveCharacterTextSplitter`, com blocos de aproximadamente 1.000 caracteres e sobreposição de 200 caracteres; (b) gerar embeddings com o modelo `nomic-embed-text`; (c) permitir a exclusão de documentos individuais, em conformidade com o direito ao

esquecimento previsto na LGPD; e (d) executar a remoção física dos vetores correspondentes, no caso de solicitação administrativa.

Em complemento, o isolamento por workspace é tratado como um requisito de segurança, com vistas a impedir que uma consulta recupere documentos de um domínio de conhecimento alheio.

3.4 Agente de auditoria e conformidade

O agente de auditoria, implementado em `audit.py`, é materializado como um middleware do Starlette que intercepta todas as requisições HTTP. Antes do despacho da requisição, o middleware serializa o corpo JSON, submete-o à função `mask_sensitive_data`, que aplica expressões regulares para identificação de CPF, CNPJ, e-mail, telefone e números de cartão de crédito, e registra uma linha estruturada no arquivo `audit.log`. Especificamente, cada linha contém carimbo de data e hora em UTC, endereço IP de origem, método HTTP, caminho, código de status, corpo já mascarado e identificador do usuário.

Dessa forma, o próprio log de auditoria permanece em conformidade com a LGPD, uma vez que não armazena dados pessoais em texto claro. Por sua vez, essa abordagem resolve o paradoxo da auditoria em sistemas de IA, pois permite o rastreamento integral das ações e da utilização do sistema para fins de governança corporativa, garantindo simultaneamente que o próprio repositório de auditoria não se torne um passivo de vazamento de dados pessoais.

3.5 Agente de desempenho

O agente de desempenho, implementado em `performance.py`, consiste em um coletor thread-safe em memória que armazena um anel limitado de registros de métricas. Externamente, o agente expõe três endpoints: `GET /api/metrics`, que devolve um snapshot em JSON com os percentis P50, P95 e P99 de latência por rota, além do histograma de modelos servidos nos últimos cinco minutos; `GET /api/metrics/stream`, que transmite o snapshot a cada segundo por SSE, consumido pelo painel em `/metrics`; e `POST /api/metrics/clear`, reservado a administradores.

Tal agente torna o comportamento do roteador e da cascata observável de forma explícita, na medida em que o painel exibe, em tempo real, qual tier efetivamente respondeu à última consulta e em qual proporção o roteador precisou acionar o fallback.

3.6 Modo soberano offline

A flag de ambiente `CERRA_REMOTE_MODE` alterna entre dois backends de persistência e autenticação por trás de uma mesma interface `firestore_client`. No modo offline soberano, padrão do sistema, a persistência é realizada em um arquivo SQLite local (`./local_store.db`), cuja interface reproduz os métodos da biblioteca do Firestore, de modo que o código de aplicação permaneça inalterado.



Figura 2. Visão do frontend da plataforma.

Da mesma forma, a autenticação, nesse modo, é implícita e atribui ao usuário local o papel de administrador, sem apresentação de qualquer prompt de credencial. No modo remoto, ativado quando o sistema é exposto por meio de um túnel, a mesma interface passa a delegar a autenticação ao Firebase e a persistência ao Firestore, mantendo-se um fallback por token de rede local.

4. Implementação

A implementação reside em um único projeto Python, no qual os módulos se relacionam um a um com os agentes descritos na Seção 3. Conforme detalhado adiante, a Tabela 1 apresenta a correspondência entre módulos, agentes e tamanho aproximado de código.

Módulo	Agente	Linhas (aprox.)	Função
main.py	Orquestrador	1.050	Rotas FastAPI, CORS, frontend estático
llm_router.py	Roteador e cascata	390	Heurística e HTTP ao Ollama
workspaces.py	RAG	75	ChromaDB por workspace
audit.py	Auditoria	105	Middleware e mascaramento
performance.py	Desempenho	220	Coletor e SSE
auth.py	Autenticação	—	Firebase e JWT (somente modo remoto)

firestore_client.py	Persistência	—	Adapter SQLite e Firestore
config.py	Configuração	—	Variáveis de ambiente com defaults
desktop_app.py	Empacotamento	—	pywebview, pythonw, ícone na taskbar
install_shortcuts.py	Instalador	—	Atalhos .lnk com ícone da instituição
static/app.js + index.html	Frontend	—	Interface estilo chat, pílula de roteamento

Tabela 1. Correspondência entre módulos, agentes e tamanho aproximado de código

Nota. Linhas aproximadas referem-se à implementação no momento da redação deste artigo.

O stack completo é executado em Windows 10 ou 11, com Python 3.11, Ollama versão 0.3 ou superior, e um único `pip install -r requirements.txt`. De forma complementar, a containerização é viabilizada pelo `docker-compose up --build`, com a mesma alternância entre os modos offline e remoto. Os modelos, por seu turno, são baixados por meio do mecanismo padrão do Ollama (Ollama, 2024), a partir dos seguintes identificadores: `nomic-embed-text` para embeddings, e o conjunto de cascata formado por `qwen2.5:1.5b`, `gemma3:latest`, `gemma4:latest` e `llama3.1:8b`.

Na camada de apresentação, o frontend, construído em HTML, CSS e JavaScript sem dependência de frameworks reativos, adota o estilo de chat com uma barra lateral de sessões, um cabeçalho com seletor de workspace e a superfície de conversação. No cabeçalho, uma pílula de roteamento atualiza a cada resposta para indicar, por exemplo, "Tier 2 (gemma3, 4,3 B) · 1.240 ms", o que torna a natureza agêntica do sistema visível para usuários não técnicos.

5. Cenários de Demonstração

Quatro cenários serão executados ao vivo durante a sessão de demonstração, com vistas a evidenciar a generalidade da arquitetura em domínios de aplicação diversos. Quanto à reprodutibilidade, cada cenário é reprodutível a partir de uma instalação limpa, com os quatro modelos de cascata previamente baixados no Ollama.

5.1 Cenário A: escalonamento progressivo de tiers

O apresentador carrega um regulamento interno em um workspace recém-criado, denominado `politicarh`, e, em seguida, formula três consultas de complexidade crescente. Em primeiro lugar, a primeira, "Olá", é encaminhada ao Tier 1 (`qwen2.5:1.5b`), sem necessidade de RAG. Em segundo lugar, a segunda, sobre o número de dias de férias previsto em determinada cláusula, eleva o contexto e aciona o Tier 2 (`gemma3, 4,3 B`). Em terceiro lugar, a terceira, que solicita a comparação entre duas cláusulas do regulamento acompanhada de análise crítica, ativa a palavra-chave correspondente e o

limiar de contexto, o que resulta no acionamento do Tier 3-Raciocínio (gemma4, 8 B).

Ao longo do cenário, a pílula de roteamento muda de forma visível, e o log de auditoria é inspecionado para confirmar o mascaramento dos corpos de requisição.

5.2 Cenário B: override e observabilidade

O apresentador aciona o menu de engrenagem e força o Tier 3-Código (llama3.1:8b), para então formular uma pergunta de programação relacionada à validação de dados, por exemplo, uma função que valide um identificador fiscal. Nessa configuração, o sistema responde com o modelo forçado, e o painel de desempenho registra o aumento correspondente de latência.

Em seguida, o apresentador induz uma falha no modelo forçado, por exemplo, interrompendo o processo subjacente, de modo a evidenciar a cascata: o próximo modelo mais leve assume a resposta de forma transparente, e a lista de tentativas é registrada no agente de desempenho.

5.3 Cenário C: mascaramento e direito ao esquecimento

Uma requisição com CPF, e-mail e telefone em formato válido é submetida ao sistema. Conforme esperado, a linha correspondente no log de auditoria e a visualização em /admin/audit são inspecionadas, e os dados aparecem como MASKED. Na sequência, o apresentador aciona a exclusão física de um documento por meio de DELETE /admin/workspace/{slug}/document/{filename} e demonstra que os blocos vetoriais correspondentes foram removidos do ChromaDB.

5.4 Cenário D: workspaces como fronteira de segurança

Dois workspaces, politicas-rh e seguranca-ti, são populados com documentos disjuntos. A título ilustrativo, uma consulta plausível em um dos workspaces é então submetida no outro, e o sistema reporta corretamente a ausência de informação. Em contrapartida, a mesma consulta, desta vez no workspace correto, produz resposta fundamentada, com citação do bloco recuperado.

Diante do exposto, o cenário evidencia que o isolamento por workspace constitui uma fronteira de segurança concreta, e não apenas um rótulo de interface, e demonstra a viabilidade de se operar a mesma arquitetura em domínios de conhecimento distintos.

6. Discussão

Esta seção sintetiza observações empíricas acumuladas durante o período de uso produtivo do sistema, com vistas a fundamentar a discussão proposta.

Em primeiro lugar, a distribuição efetiva de consultas por tier revela um padrão dominado por consultas de baixa complexidade. Especificamente, estima-se que aproximadamente 70% das consultas sejam atendidas pelo Tier 1, 20% pelo Tier 2 e 10% pelos tiers 3, em consonância com o perfil de demandas típico de uma organização que combina atendimento a usuários finais e suporte técnico interno. Tal distribuição sugere

que o roteador heurístico é capaz de aproximar a complexidade observada das consultas de modo compatível com a literatura de cascadeamento de modelos (Chen et al., 2023; Ding et al., 2024). Essa distribuição valida a premissa de redução de custos operacionais e de infraestrutura. Ao resolver 70% das demandas com um modelo de 1.5B parâmetros, o sistema libera recursos de GPU para as tarefas de raciocínio complexo e código (tiers 3), viabilizando a operação de um assistente organizacional em hardware local padrão (commodity hardware), sem a necessidade de dispendiosos clusters de processamento ou assinaturas de serviços em nuvem.

Em segundo lugar, a cascata raramente é acionada, mas revela-se útil nos casos em que é. Em uma janela de trinta dias, menos de 1% das consultas exigiram fallback, e a quase totalidade desses casos correspondeu a esgotamentos de memória do Tier 3-Raciocínio em contextos longos. Na ausência da cascata, essas falhas se traduziriam em erros visíveis ao usuário, em vez de fallback transparente.

Em terceiro lugar, o mascaramento de dados sensíveis não constitui um recurso acessório, mas um pré-requisito operacional. Na prática, o log de auditoria é rotineiramente consultado em procedimentos regulatórios e em auditorias internas, de modo que o armazenamento de dados pessoais em texto claro nesse artefato comprometeria a finalidade do próprio registro.

Por fim, a execução totalmente local não é uma decisão técnica isolada, mas o que torna a arquitetura legalmente implantável. Em corroboração, a literatura aponta que a combinação entre inferência em borda e adesão a regulações de proteção de dados tem se tornado um requisito para setores sensíveis (Carlini et al., 2021). Em organizações de diferentes portes e segmentos, a combinação de execução local, mascaramento de dados sensíveis e empacotamento em executável de distribuição é o que viabiliza a operação cotidiana.

A arquitetura proposta também suscita questões de pesquisa alinhadas aos tópicos de interesse do WESAAC 2026 (Sociedade Brasileira de Computação [SBC], 2026). Em primeiro lugar, a substituição do roteador heurístico por um classificador aprendido, treinado sobre os registros do agente de desempenho, é uma evolução natural. Em segundo lugar, a introdução de um protocolo de negociação entre o roteador e os agentes LLM, no qual o modelo pode sinalizar confiança insuficiente e solicitar escalonamento, transformaria a cascata em uma interação multi-agente plena. Por fim, a construção de um harness de avaliação compartilhado, sob o tópico "Evaluation of LLMs/agent systems", permitiria a terceiros a medição de acurácia de roteamento, frequência de fallback e revocação do mascaramento sobre um snapshot congelado do sistema.

7. Conclusão

Este trabalho apresentou uma arquitetura multi-agente de LLMs executada integralmente em ambiente local e organizada em torno de um roteador dinâmico em cascata. Em sua composição, a arquitetura combina um agente roteador heurístico, quatro agentes LLM especializados, um agente de RAG isolado por workspace, um agente de auditoria e um agente de desempenho, e encontra-se em uso produtivo em uma organização de referência, embora não esteja vinculada a qualquer setor de aplicação específico.

Externamente, a demonstração da proposta contempla quatro cenários

reprodutíveis, que evidenciam o escalonamento progressivo de tiers, o comportamento de fallback, o mascaramento de dados sensíveis e o papel do isolamento por workspace como fronteira de segurança, com exemplos distribuídos entre domínios de aplicação distintos.

Como trabalhos futuros, pretende-se (a) substituir o roteador heurístico por um classificador aprendido, treinado a partir do fluxo de auditoria e desempenho; (b) introduzir um protocolo de negociação entre o agente roteador e os agentes LLM, no qual sinais de baixa confiança possam disparar o escalonamento de forma explícita; (c) publicar um harness de avaliação que permita a terceiros mensurar a acurácia do roteamento, a frequência de fallback e a revocação do mascaramento LGPD sobre um snapshot congelado do sistema; e (d) investigar um modo de federação leve, no qual múltiplas organizações compartilhem o agente roteador e o agente de desempenho, mas mantenham os agentes de RAG e auditoria plenamente locais. Diante disso, espera-se que a disponibilização da arquitetura e de seu roteiro de demonstração contribua para a discussão, na comunidade do WESAAC, sobre a implantação responsável de sistemas agênticos baseados em LLM em ambientes organizacionais regulados.

Referências

- Bordini, R. H., Hübner, J. F. and Wooldridge, M. (2007) *Programming Multi-Agent Systems in AgentSpeak using Jason*, John Wiley & Sons.
- Brasil. (2018) "Lei nº 13.709, de 14 de agosto de 2018: Lei Geral de Proteção de Dados Pessoais (LGPD)", *Diário Oficial da União*, https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm
- Carlini, N., Ippolito, D., Jagielski, M., Lee, K., Tramèr, F. and Zhang, C. (2021) "Extracting Training Data from Large Language Models", In: *Proceedings of the 30th USENIX Security Symposium*, USENIX Association, p. 2633–2650.
- Chen, L., Zaharia, M. and Zou, J. (2023) "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance", *IEEE Transactions on Knowledge and Data Engineering*, 36(7), p. 3299–3313.
- Chroma. (2024) Chroma documentation, <https://www.trychroma.com>
- Conselho da União Europeia. (2016) "Regulamento (UE) 2016/679 do Parlamento Europeu e do Conselho, de 27 de abril de 2016, relativo à proteção das pessoas singulares no que diz respeito ao tratamento de dados pessoais e à livre circulação desses dados (GDPR)", *Jornal Oficial da União Europeia*, <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- Dignum, V. (2004) *A model for organizational interaction: Based on agents, founded in logic*, Tese de doutorado, Utrecht University.
- Ding, D., Mallick, A., Wang, C. et al. (2024) "Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing", In: *Proceedings of the 12th International Conference on Learning Representations*, OpenReview.
- Estado da Califórnia. (2018) *California Consumer Privacy Act (CCPA)*, California Legislative Information,

https://leginfo.legislature.ca.gov/faces/codes_displayText.xhtml?division=3.&part=4.&lawCode=CIV

- Gao, Y., Xiong, Y., Gao, X. et al. (2023) "Retrieval-Augmented Generation for Large Language Models: A Survey", arXiv preprint arXiv:2312.10997.
- LangChain. (2024) LangChain documentation, <https://python.langchain.com>
- Lewis, P., Perez, E., Piktus, A. et al. (2020) "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", In: Advances in Neural Information Processing Systems, Curran Associates, p. 9459–9474.
- Mialon, G., Dessì, R., Lomeli, M. et al. (2023) "Augmented Language Models: A Survey", Transactions on Machine Learning Research, 14(8), p. 1–36.
- Ollama. (2024) Get up and running with large language models locally, <https://ollama.com>
- Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P. and Bernstein, M. S. (2023) "Generative Agents: Interactive Simulacra of Human Behavior", In: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, ACM, p. 1–22.
- Rao, A. S. and Georgeff, M. P. (1995) "BDI Agents: From Theory to Practice", In: Proceedings of the 1st International Conference on Multi-Agent Systems, AAAI Press, p. 312–319.
- Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th edition, Pearson.
- Schick, T., Dwivedi-Yu, J., Dessì, R. et al. (2023) "Toolformer: Language Models Can Teach Themselves to Use Tools", In: Proceedings of the 37th Conference on Neural Information Processing Systems, Curran Associates.
- Schuster, T., Fisch, A., Gupta, J. et al. (2021) "Confident Adaptive Language Modeling", In: Advances in Neural Information Processing Systems, Curran Associates, p. 17456–17472.
- Sociedade Brasileira de Computação. (2026) WESAAC 2026: Call for papers, <https://bracis.com.br/wesaac2026>
- Wooldridge, M. (2009) An Introduction to Multiagent Systems, 2nd edition, John Wiley & Sons.
- Yao, S., Zhao, J., Yu, D. et al. (2023) "ReAct: Synergizing Reasoning and Acting in Language Models", In: Proceedings of the 11th International Conference on Learning Representations, OpenReview.
- Wei, J., Wang, X., Schuurmans, D. et al. (2022) "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", In: Advances in Neural Information Processing Systems, Curran Associates, p. 24824–24837.